

OOP

programare orientată pe obiecte

CURS06: POLIMORFISM. DOUBLE DISPATCH. VISITOR

Alexandru Olteanu
{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
Universitatea Politehnica Bucureşti

POLIMORFISM

alexandru.olteanu@upb.ro



Polimorfismul este caracteristica unei entitati de a avea forme diferite (structural, comportamental etc.) in contexte diferite.

```
class Vehicle
interface Diesel
interface Electric
interface PublicTransport
class Hybrid extends Vehicle implements Electric , Diesel
class Bus extends Vehicle implements Diesel , PublicTransport
```

- A Bus IS-A Vehicle
- A Bus IS-A Diesel
- A Bus IS-A PublicTransport
- A Bus IS-A Bus
- A Bus IS-A Object

Daca avem acelasi membru / metoda in mai multe "fatete" ale unui obiect, care se foloseste?

Ad-hoc polymorphism (...) allows a polymorphic value to exhibit different behaviors when viewed at different types. The most common example of ad-hoc polymorphism is overloading, which associates a single function symbol with many implementations; the compiler (or the runtime system, depending on whether overloading resolution is static or dynamic) chooses an appropriate implementation for each application of the function, based on the types of the arguments.

B. Pierce, "Types and Programming Languages", MIT Press

alexandru.olteanu@upb.ro

- static polymorphism
- dynamic polymorphism
- parametric polymorphism -> generics

alexandru.olteanu@upb.ro

Java:
Static (compile time) Polymorphism = Overloading

```
class Profesor {
    public String explicaBazaDeDate (Copil c) {
         return "dulap cu sertare";
    }

    public String explicaBazaDeDate (Student s) {
        return "colecție organizată de informații";
    }
}
...
public static void main ( String args []) {
        Copil c = new Copil();
        Profesor p = new Profesor();
        p.explicaBazaDeDate(c);
}
```

Java: Dynamic (run time) Polymorphism = Overriding

```
public interface Problema { String solvingTime(); }
public class ProblemaSimpla {
    String solvingTime() {
        return "5 minute";
    }
}
public class ProblemaGrea {
    String solvingTime() {
        return "5 ore";
    }
}
...
public static void main ( String args []) {
    Problema s = new ProblemaSimpla();
    System.out.println(s.solvingTime());
}
```

Java:
Dynamic (run time) Polymorphism = Overriding

Late binding, or dynamic binding is a computer programming mechanism in which the method being called upon an object or the function being called with arguments is looked up by name at runtime

* Private, final and static methods and variables use static binding

Java: Atentie! dynamic binding se aplica la metode, pentru campuri se face static binding

```
class Evaluator {
    public int nr_teste = 5;
}
class EvaluatorGreu extends Evaluator {
    public int nr_teste = 10;
}
...
Evaluator e = new EvaluatorGreu();
System.out.println(e.nr_teste);
```

```
class Evaluator {
    public int nr_teste = 5;
}
class EvaluatorGreu extends Evaluator {
    public EvaluatorGreu() {
        nr_teste = 10;
    }
}
...
```

```
Evaluator e = new EvaluatorGreu();
System.out.println(e.nr_teste);
```

: Polimorfism. Visitor

ale

C++:

Se va face dynamic binding (la run-time) doar pentru functiile marcate ca fiind virtuale Daca o functie este supraincarcata fara a marcata drept virtuala, se va face static binding (la compilare)

Despre functii virtuale in C++

Exemplu de supraincarcarea operatorilor in C++

DOUBLE DISPATCH

alexandru.olteanu@upb.ro

The problem with Single Dispatch

```
interface Problema { }
class Simpla implements Problema { }
class Grea implements Problema { }

interface Evaluator { void eval(Simpla s); void eval(Grea g); }
class Automat implements Evaluator {
    public void eval(Simpla s) { sout("10"); }
    public void eval(Grea g) { sout("9"); }
}
class Manual implements Evaluator {
    public void eval(Simpla s) { sout("8"); }
    public void eval(Grea g) { sout("7"); }
}
Problema s = new Simpla(); Problema g = new Grea();
Evaluator a = new Automat(); Evaluator m = new Manual();
a.eval(s); a.eval(g); m.eval(a); m.eval(g);
```

Mixing the Second Dispatch on top

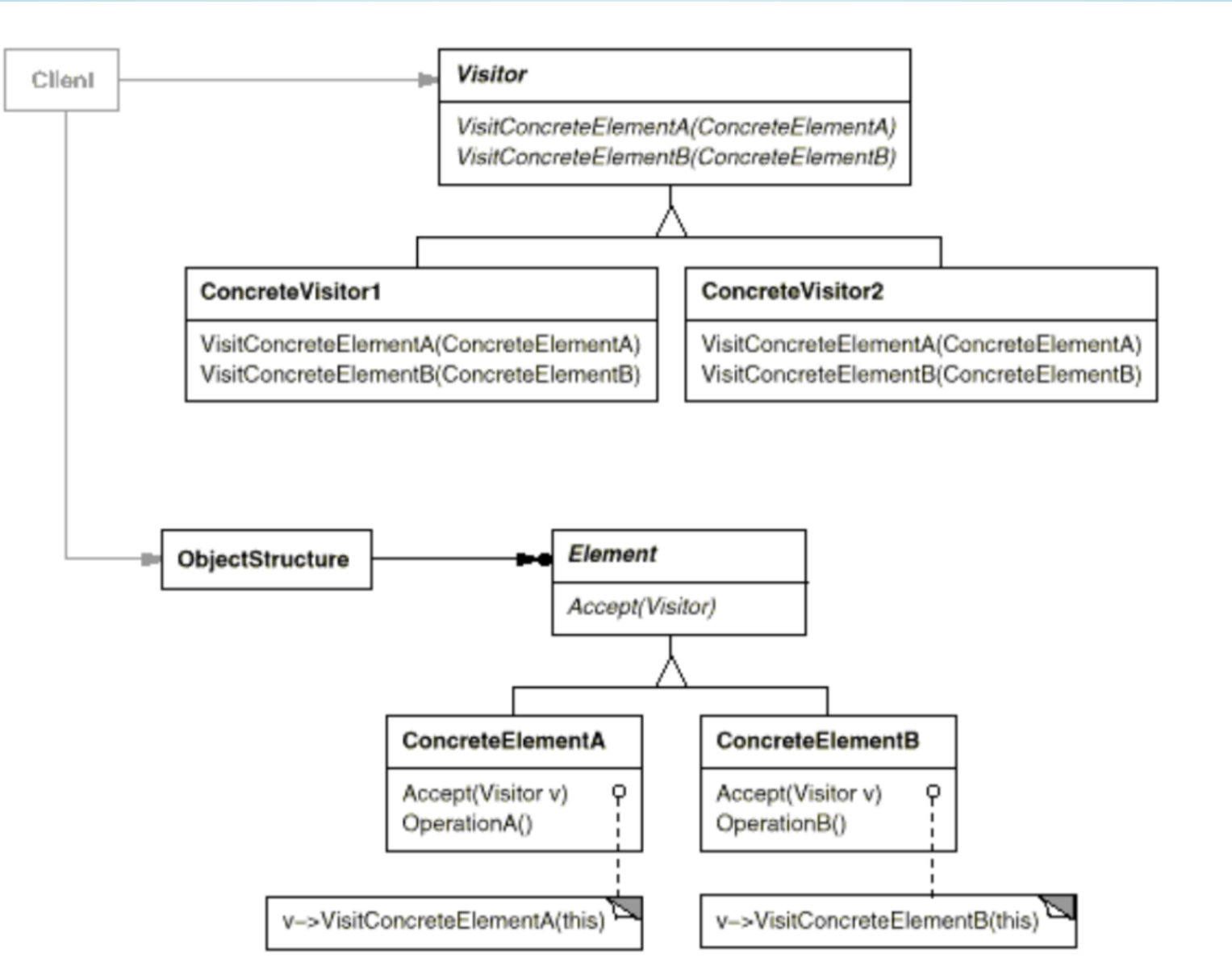
```
interface Problema { void submit(Evaluator e); }
class Simpla implements Problema {
    public void submit(Evaluator e) {
        e.eval(this);
    }
}
class Grea implements Problema {
    public void submit(Evaluator e) {
        e.eval(this);
    }
}
// Evaluator et co. raman la fel
...
Problema s = new Simpla(); Problema g = new Grea();
Evaluator a = new Automat(); Evaluator m = new Manual();
s.submit(a); g.submit(a); s.submit(m); g.submit(m);
```

VISITOR

alexandru.olteanu@upb.ro

Visitor represents an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.

- o structura contine obiecte de clase variate, asupra carora vrem sa aplicam operatii ce depind de clasele lor concrete
- operatiile pe care vrem sa le aplicam sunt diferite, fara legatura intre ele si ar trebui sa e usor extensibile
- clasele ce denesc structura de obiecte se schimba rar, dar vrem sa denim operatii noi usor (daca si clasele ce denesc structura de obiecte se schimba des, poate e mai bine sa denim operatiile in ele)


Client
Visitor
VisitConcreteElementA(ConcreteElementA)
VisitConcreteElementB(ConcreteElementB)
ConcreteVisitor1
VisitConcreteElementA(ConcreteElementA)
VisitConcreteElementB(ConcreteElementB)
ConcreteVisitor2
VisitConcreteElementA(ConcreteElementA)
VisitConcreteElementB(ConcreteElementB)
ObjectStructure
Element
Accept(Visitor)
ConcreteElementA
Accept(Visitor v)
OperationA()
ConcreteElementB
Accept(Visitor v)
OperationB()
v->VisitConcreteElementA(this)
v->VisitConcreteElementB(this)

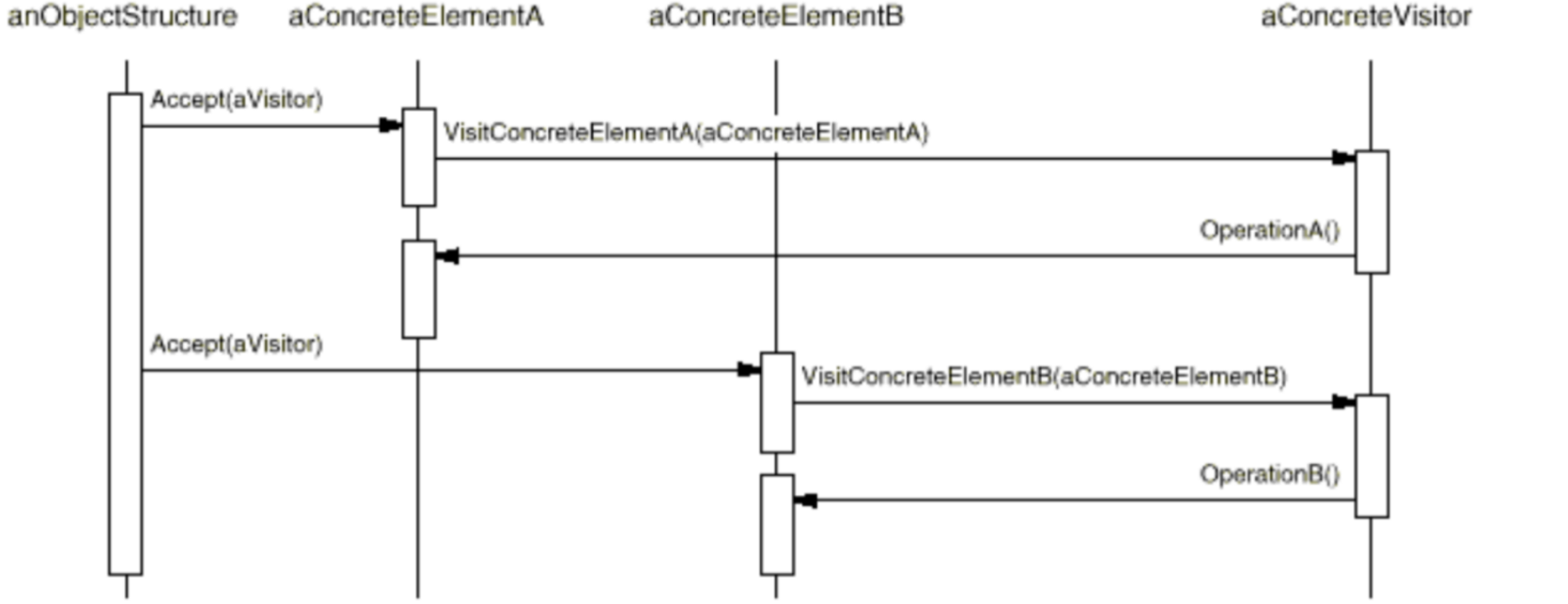
	ConcreteVisitor1	ConcreteVisitor2
ConcreteElementA	VisitConcreteElementA	VisitConcreteEleme
ConcreteElementB	VisitConcreteElementB	VisitConcreteEleme

Un client trebuie sa instantieze cate un vizitator concret pentru ecare operatie si sa traverseze structura de obiecte vizitand ecare obiect cu ecare vizitator concret.

Cum se face traversarea?

- de catre structura de obiecte
- de catre vizitator
- intr-un obiect iterator separat

Vizitatorul concret ofera un context in care sa se desfasoare operatia, acumuleaza stare si poate apela metode din obiecte:



* E. Gamma, R. Helm, R. Johnson, J. Vlissides "Design Patterns: Elements of Reusable Object-Oriented Software"

Exercitii:

- Incercati sa adaugati o noua clasa de vizitator
- Incercati sa adaugati o noua clasa de obiecte vizitate

- adaugarea de operatii este simpla (adaugarea unui nou vizitator vs modicarea fiecarei clase de obiecte vizitate)
- adaugarea de noi clase de obiecte vizitate este mai dificila
- vizitarea structurii de obiecte vizitate se poate face in diferite moduri, dar in forma cea mai pura de Visitor, spre deosebire de Iterator,
- obiectele vizitate nu trebuie neaparat sa instantieze dintr-o ierarhie de clase
- vizitatorul poate acumula stare
- implementarea de operatii in obiectele vizitate care sa permita vizitatorului sa isi faca treaba poate compromite incapsularea in clasele de

poate compromite încapsularea în clasele de obiecte vizitate